webdriver — browser automation with Babet

`webdriver.lua` is a **W3C WebDriver Classic** client for Babet 2.9.0 or newer. Since version 2.0.0, `webdriver_bidi.lua` adds **WebDriver BiDi** on top of the native WebSocket client provided by Babet 2.22.0 or newer. The library drives Firefox, Chrome and Chromium through geckodriver or chromedriver. Babet is available from its official GitHub repository.

Table of contents

Installation

The files `webdriver.lua`, `webdriver_version.lua`, `driver_manager.lua` and, depending on the selected mode, `webdriver_worker.lua`, `webdriver_bidi.lua` and `webdriver_bidi_worker.lua` must be reachable through `package.path`.

```
local webdriver = require("webdriver")
```

The library requires:

- Babet 2.9.0 or newer for WebDriver Classic;
- Babet 2.22.0 or newer for WebDriver BiDi and the complete 2.0 test suite;
- Linux;
- Firefox, Chrome or Chromium;
- geckodriver or chromedriver, either installed or downloadable.

It does not require `tar`, `unzip`, `curl`, `base64` or an external shell.

Starting a new project

Direct project: required files

To drive a browser from the main script, copy only:

```
my-project/
├── webdriver.lua
├── webdriver_version.lua
├── driver_manager.lua
└── main.lua
```

`webdriver.lua` loads `webdriver_version.lua` and `driver_manager.lua` internally. All three modules are therefore required even when geckodriver or chromedriver is already installed.

When the modules are next to `main.lua`, loading is direct:

```lua
local webdriver = require("webdriver")
```

Project using a worker

To host the WebDriver session inside a worker and communicate through channels, add the proxy module:

```
my-project/
├── webdriver.lua
├── webdriver_version.lua
├── driver_manager.lua
├── webdriver_worker.lua
└── main.lua
```

The main script then loads:

```lua
local webdriver_worker = require("webdriver_worker")
```

For direct BiDi, add `webdriver_bidi.lua`. For a dedicated BiDi worker, also add `webdriver_bidi_worker.lua`. A Classic session hosted by `webdriver_worker.lua` uses the same module when `session:bidi()` is called.

`tools/check_env.lua` is an optional diagnostic tool. The `tests/`, `examples/` and `docs/` directories, as well as `build_docs.sh` and the `run_*.sh` scripts, are not required in an application project.

Location of the Babet executable

The repository's `bin/` directory is only a convenient convention. Babet may be installed anywhere. It can be downloaded or built from its GitHub repository, and babet-webdriver requires **2.9.0** or newer for the Classic client. WebDriver BiDi requires **Babet 2.22.0** or newer.

Project-local executable:

```
my-project/
├── bin/babet
├── webdriver.lua
├── webdriver_version.lua
├── driver_manager.lua
└── main.lua
```

```
./bin/babet main.lua
```

Babet installed in `PATH`:

```
babet main.lua
```

A shebang also allows:

```
#!/usr/bin/env babet
```

```
chmod +x main.lua
./main.lua
```

Babet at an arbitrary path:

```
/opt/babet/bin/babet main.lua
```

Inside the babet-webdriver repository, `run_tests.sh`, `run_smoke.sh`, `run_worker_smoke.sh` and `run_all_tests.sh` locate the executable in this order:

1. the path provided through `BABET`;
2. `bin/babet`;
3. a `babet` command available in `PATH`.

Example:

```
BABET=/opt/babet/bin/babet ./run_tests.sh
```

To run the complete validation campaign with a single command:

```
./run_all_tests.sh
```

The script enables headless mode by default, keeps the full progress visible in the terminal and copies the exact same output to `babet-webdriver-tests.txt`. The log is first written to a unique temporary file in the destination directory and is atomically published only when the campaign finishes. An existing complete log is therefore never truncated while tests are running, and accidentally parallel campaigns never mix their contents. If several campaigns target the same path, the one that finishes last simply becomes the current log. A completed campaign, whether successful or failed, replaces the previous log so it can be shared as-is. Use `HEADLESS=0` to display the browsers, or `TEST_LOG=/path/test.txt` to select another file.

Complete minimal example

```lua
#!/usr/bin/env babet

local webdriver = require("webdriver")

local driver, err = webdriver.firefox({ headless = true })
if not driver then
    io.stderr:write(tostring(err), "\n")
    os.exit(1)
end

local ok, run_err = xpcall(function()
    assert(driver:open("https://example.com"))
    print(assert(driver:title()))

    local heading = assert(driver:css("h1"))
    print(assert(heading:text()))
end, debug.traceback)

local quit_ok, quit_err = driver:quit()
if not quit_ok then
    io.stderr:write("WebDriver shutdown: ", tostring(quit_err), "\n")
end

if not ok then
    io.stderr:write(tostring(run_err), "\n")
    os.exit(1)
end
```

Modules in a subdirectory

To keep the modules under `lib/` instead:

```
my-project/
├── lib/
│   ├── webdriver.lua
│   ├── webdriver_version.lua
│   ├── driver_manager.lua
│   └── webdriver_worker.lua
└── main.lua
```

Add the directory before the first `require`:

```lua
package.path = "./lib/?.lua;" .. package.path
```

```lua
local webdriver = require("webdriver")
```

Creating a session

Firefox

```lua
local driver, err = webdriver.firefox({
    headless = true,
})
assert(driver, err)
```

Chrome or Chromium

```lua
local chrome = assert(webdriver.chrome({ headless = true }))
local chromium = assert(webdriver.chromium({ headless = true }))
```

The convenience constructors never modify the caller's option table.

Generic form

```lua
local driver = assert(webdriver.start({
    browser = "firefox",
    headless = true,
}))
```

Start options

Browser and capabilities

Option	Type	Default	Description
`browser`	string	`firefox`	`firefox`, `chrome`, `chromium`
`headless`	boolean	`false`	adds the browser-specific headless arg
`args`	dense array	`{}`	browser command-line arguments
`binary`	string	automatic for Chromium	explicit browser executable
`user_data_dir`	string	absent	explicit Chrome/Chromium profile
`window_size`	`{w,h}`	absent	initial window size
`accept_insecure_certs`	boolean	`false`	corresponding W3C capability
`bidi`	boolean	`false`	requests `webSocketUrl` and negotiates

```lua
local driver = assert(webdriver.firefox({
    headless = true,
    args = { "--private-window" },
    binary = "/usr/lib/firefox/firefox",
    window_size = { 1600, 1000 },
    accept_insecure_certs = false,
}))
```

For `webdriver.chromium()`, the executable is searched in `PATH` under the names `chromium` and then `chromium-browser`. For `webdriver.chrome()`, the library tries `google-chrome-stable`, `google-chrome` and then `chrome`, but lets ChromeDriver apply its own normal detection when none of those names is found.

Chromium distributed as a Snap

ChromeDriver normally creates a temporary profile. With Chromium installed as a Snap, that profile may be outside the filesystem space visible through Snap confinement, which can cause

`DevToolsActivePort file doesn't exist`.

When the detected executable is under `/snap/bin/` or `/snap/chromium/`, the library therefore creates a temporary profile under:

```
~/snap/chromium/common/babet-webdriver/profiles/
```

That profile is passed through `--user-data-dir=...` and removed from `driver:quit()`. The library does not add `--no-sandbox`; the browser sandbox remains enabled.

An explicit profile is still supported:

```
local driver = assert(webdriver.chromium({
    headless = true,
    user_data_dir = os.getenv("HOME") .. "/snap/chromium/common/my-profile",
}))
```

`user_data_dir` cannot be combined with a manually supplied `--user-data-dir=...` argument in `args`. A profile supplied by the caller is never removed automatically.

External driver

Option	Type	Default	Description
`driver_path`	string	PATH lookup	explicit driver path
`auto_install`	boolean	`true`	installs the driver when missing
`trust_on_first_use`	boolean	`false`	permits the first unpinned artifact
`expected_sha256`	hexadecimal string	absent	expected archive hash
`platform`	string	automatic	forced platform key
`cache`	string	default cache	cache directory
`force_driver_download`	boolean	`false`	forces reinstallation

Port and connection

Option	Type	Default	Description
`port`	integer	automatic	WebDriver server port
`port_attempts`	integer	`5`	retries for automatic port selection
`attach`	boolean	`false`	uses an external driver ready to create a new session
`start_timeout`	seconds	`15`	startup time budget
`status_timeout`	seconds	`1`	timeout for each `/status` probe
`poll_interval`	seconds	`0.1`	delay between probes

Without an explicit port, the library asks the kernel for a free port. The temporary socket must then be closed before the driver starts, so a small race window remains. `port_attempts` lets the library retry when another process takes the port in that interval.

In `attach` mode, the external driver must answer `/status` with `ready=true`, so it must be available to create **a new session**. This mode does not resume an existing Selenium session. In particular, geckodriver normally reports `ready=false` while already occupied; babet-webdriver then reports it as reachable but unavailable instead of misdiagnosing a network failure.

In `attach` mode, the historical defaults are preserved:

- Firefox: 4444;
- Chrome/Chromium: 9515.

```
local driver = assert(webdriver.firefox({
    attach = true,
    port = 4444,
}))
```

HTTP, screenshots and logs

Option	Default	Description
`request_timeout`	120 s	WebDriver command timeout
`max_body_size`	64 MiB	maximum HTTP response body
`screenshot_max_size`	64 MiB	limit after Base64 decoding
`screenshot_permissions`	0644	permissions of the published PNG
`screenshot_durable`	`true`	fsyncs the file and directory
`print_max_size`	64 MiB	limit after PDF Base64 decoding
`print_permissions`	0644	permissions of the published PDF
`print_durable`	`true`	fsyncs the PDF and directory
`log_path`	automatic	exact log file
`log_dir`	cache/logs	automatic log directory
`log_append`	`true`	appends instead of truncating
`log_permissions`	0600	creation permissions
`terminate_grace`	2 s	grace period before SIGKILL

Options are strict. A typo such as `timeot = 5` immediately raises a Lua programming error.

Driver lifecycle

When the library starts the driver, it keeps the object returned by `babet.spawn()`:

```
print(driver:pid())
print(driver:port())
print(driver:log_path())
print(driver:is_running())
```

The streams are configured as follows:

- stdin is connected to `/dev/null`;
- stdout is redirected to the log file;
- stderr is merged into stdout.

No shell interprets the executable path or its arguments.

Closing

```
local ok, err = driver:quit()
assert(ok, err)
```

`quit()`:

1. requests deletion of the W3C session;
2. sends SIGTERM to the driver process group;
3. waits for `terminate_grace`;
4. uses SIGKILL when necessary;
5. closes the Babet descriptors.

The method is idempotent. `driver:close()` is an alias. The object also exposes an `__close` metamethod for Lua to-be-closed variables.

Navigation

```
assert(driver:open("https://example.com"))
print(assert(driver:url()))
print(assert(driver:title()))
print(assert(driver:source()))
assert(driver:back())
```

```lua
assert(driver:forward())
assert(driver:refresh())
```

Locating elements and waiting

Strategies

```lua
driver:find("main h1")                          -- CSS
driver:find("//main//h1", { by = "xpath" })
driver:find("submit", { by = "id" })
driver:find("email", { by = "name" })
driver:find("button", { by = "tag" })
driver:find("active", { by = "class" })
driver:find("Home", { by = "link" })
driver:find("Hom", { by = "plink" })
```

Convenience methods:

```lua
driver:css(selector)
driver:xpath(selector)
driver:id(value)
driver:name(value)
driver:tag(value)
```

`id`, `name` and `class` are converted to correctly escaped CSS selectors.

One element

```lua
local element, err = driver:css("h1")
assert(element, err)
```

When no element matches, the server returns the W3C `no such element` error.

Multiple elements

```lua
local elements = assert(driver:find_all("article"))
for _, element in ipairs(elements) do
    print(assert(element:text()))
end
```

An empty array is a normal successful result.

Checking presence

```lua
local exists, err = driver:exists("#optional")
assert(exists ~= nil, err)
if exists then
    print("present")
end
```

Only `no such element` becomes `false`. A network failure, a closed session or a dead driver remains an error.

Waiting for a state

```lua
local button = assert(driver:wait("#submit", {
    state = "clickable",
    timeout = 20,
```

```
    interval = 0.2,
}))
```

States:

- `present`: the element is found;
- `visible`: found and displayed;
- `clickable`: displayed and enabled;
- `gone`: absence is confirmed.

By default, a stale reference causes a fresh lookup. Other errors are returned immediately.

Custom wait

```
local result = assert(driver:wait_until(function()
    local value, err = driver:js("return window.appReady === true")
    if value == nil and err then return nil, err end
    return value
end, 30, 0.25))
```

A callback exception or an error returned as the second value is not silently ignored.

Elements

Simple actions

```
assert(element:click())
assert(element:clear())
assert(element:type("Hello"))
assert(element:submit())
```

`submit()` finds the enclosing form through JavaScript and prefers `requestSubmit()` when available.

Reading values

```
print(assert(element:text()))
print(assert(element:tag()))
print(assert(element:rect()).width)
print(assert(element:css("display")))
print(assert(element:property("value")))
print(assert(element:dom_attr("data-id")))
print(assert(element:attr("textContent")))
print(assert(element:value()))
print(assert(element:computed_role()))
print(assert(element:computed_label()))
```

`attr()` mirrors Selenium's practical behavior: it tries the HTML attribute and then the DOM property. If neither exists, it returns `nil` without an error. Generic JavaScript results still preserve `babet.json.null`.

Boolean states

```
local displayed, err = element:displayed()
assert(displayed ~= nil, err)
```

`displayed`, `enabled` and `selected` propagate errors. They no longer turn an error into `false`.

Nested lookup

```
local row = assert(driver:css("tr.active"))
local button = assert(row:find("button.save"))
local cells = assert(row:find_all("td"))
```

W3C Shadow DOM:

```
local host = assert(driver:css("my-component"))
local shadow = assert(host:shadow_root())
assert(webdriver.is_shadow_root(shadow))
local button = assert(shadow:find("button.primary"))
local items = assert(shadow:find_all("li"))
```

The currently active element is available through `driver:active_element()`.

JavaScript

```
local text = assert(driver:js(
    "return arguments[0].textContent",
    element
))
```

Element objects passed as arguments are serialized with the W3C element key. Elements returned by the script are rebuilt automatically, including inside a nested table.

Cyclic tables are rejected before the request is sent. A JavaScript `null` value is preserved as `babet.json.null`, keeping it distinct from an error-signalling Lua `nil`. The W3C asynchronous variant is exposed through `js_async()`:

```
local value = assert(driver:js_async([[
  const done = arguments[arguments.length - 1];
  setTimeout(() => done("ready"), 10);
]]))
```

Keyboard, mouse and wheel actions

```
assert(driver:actions()
    :move_to(element)
    :click()
    :send_keys("Hello")
    :perform())
```

Methods:

```
move_to(element [, x, y])
move_by(x, y)
click([element])
double_click([element])
context_click([element])
click_and_hold([element])
release()
key_down(character)
key_up(character)
send_keys(text)
pause(milliseconds)
scroll(delta_x, delta_y [, opts])
drag_and_drop(source, destination)
perform()
clear()
```

The builder aligns keyboard, pointer and wheel sources tick by tick. `scroll()` emits a W3C `wheel` source; its origin may be `"viewport"` or an `Element`, with optional `x`, `y` and `duration`. Coordinates, deltas and duration are integers as required by the protocol; `move_to()`, `move_by()` and `pause()` apply the same strict validation. `perform()` resets the sequences.

Special keys are exposed through `webdriver.keys`, for example:

```
webdriver.keys.ENTER
webdriver.keys.CONTROL
webdriver.keys.DELETE
webdriver.keys.F1
webdriver.keys.SEPARATOR
webdriver.keys.RIGHT_SHIFT
webdriver.keys.RIGHT_CONTROL
webdriver.keys.RIGHT_ALT
webdriver.keys.RIGHT_META
```

Windows, tabs and frames

```lua
local current = assert(driver:window())
local handles = assert(driver:windows())
assert(driver:switch(handles[#handles]))
assert(driver:switch_last())
local new_handle = assert(driver:new_tab())
local window_handle, kind = assert(driver:new_window("window"))
assert(kind == "window")
assert(driver:close_window())
```

Window rectangles:

```lua
assert(driver:set_window_rect({ x = 0, y = 0, width = 1280, height = 900 }))
local rect = assert(driver:window_rect())
assert(driver:maximize())
assert(driver:minimize())
assert(driver:fullscreen())
```

Frames:

```lua
assert(driver:frame(0))
assert(driver:frame(frame_element))
assert(driver:parent_frame())
assert(driver:top_frame())
```

Alerts, cookies and timeouts

Alerts

```lua
print(assert(driver:alert_text()))
assert(driver:alert_send("answer"))
assert(driver:accept_alert())
assert(driver:dismiss_alert())
```

Cookies

```lua
local cookies = assert(driver:cookies())
local theme = assert(driver:cookie("theme"))
assert(driver:set_cookie({ name = "theme", value = "dark" }))
assert(driver:delete_cookie("theme"))
assert(driver:clear_cookies())
```

A cookie name is encoded as a URL path segment; slashes, spaces and other characters cannot break the WebDriver path.

W3C timeouts

Values are expressed in seconds:

```
assert(driver:set_timeouts({
    implicit = 0,
    page_load = 60,
    script = 30,
}))
local timeouts = assert(driver:get_timeouts())
print(timeouts.page_load)

-- W3C allows null to remove a limit when the driver supports it.
assert(driver:set_timeouts({ script = babet.json.null }))
```

Screenshots

Without a path, the Base64 string is returned:

```
local encoded = assert(driver:screenshot())
local element_encoded = assert(element:screenshot())
```

With a path, Babet decodes the binary data and publishes the file atomically:

```
assert(driver:screenshot("captures/page.png"))
assert(element:screenshot("captures/button.png"))
```

The parent directory is created recursively when necessary. Atomic publication ensures that a reader sees either the previous complete file or the new one, never a partial PNG.

W3C printing returns a Base64 PDF or publishes it atomically when a path is provided:

```
local encoded_pdf = assert(driver:print({ orientation = "portrait" }))
assert(driver:print({
    orientation = "landscape",
    background = true,
    page = { width = 21, height = 29.7 },
    margin = { top = 1, bottom = 1, left = 1, right = 1 },
    page_ranges = { "1-2", 4 },
}, "captures/page.pdf"))
```

Automatic driver management

`driver_manager.lua` can also be used directly:

```
local manager = require("driver_manager")
local path = assert(manager.install("firefox", {
    trust_on_first_use = true,
}))
```

Resolution

- geckodriver: latest official GitHub release, unless `driver_manager.gecko_version` is forced;
- chromedriver: version matching the milestone of the Chrome or Chromium binary actually selected, unless `driver_manager.chrome_version` is forced.

Verification

The cache uses `XDG_CACHE_HOME/babet-webdriver` when available, otherwise:

```
~/.cache/babet-webdriver/pins/<driver>_<platform>_<version>.json
```

Without `HOME`, a private per-user/UID cache is selected under `/tmp` to avoid permission collisions between accounts.

Each record contains:

- the URL;
- the archive SHA-256;
- the extracted executable SHA-256;
- the version and platform.

A cached executable is reused only when its hash matches. Legacy `pins.json` files containing only a hash string are read for migration, but the new format is written under `pins/`.

TOFU

Without a known pin, installation fails by default. Setting `trust_on_first_use = true` trusts the first HTTPS download and stores its hashes.

This mode detects later modification. It does not prove that the first artifact was legitimate. For a stronger trust chain, supply `expected_sha256` from an independent channel.

For release preflight, `RUN_INSTALL_SMOKE=1 ./run_all_tests.sh` creates a fresh temporary cache and exercises the real download, extraction, chmod, hashing, atomic publication, and pin reread paths for geckodriver and chromedriver without touching the user cache.

In CI, `BABET_WEBDRIVER_GITHUB_TOKEN`, `GH_TOKEN` or `GITHUB_TOKEN` may be set to authenticate the GitHub API request used for geckodriver. If a repository- scoped GitHub Actions `GITHUB_TOKEN` is rejected by the public geckodriver repository, the request is retried anonymously. The token is never sent to other hosts.

Extraction

The archive is never loaded entirely into Lua. Babet downloads it to a temporary file, verifies the hash, then extracts into a unique temporary file before atomically publishing the executable in the cache:

- `geckodriver` for Firefox;
- `chromedriver-<platform>/chromedriver` for Chrome.

Anti-bomb limits apply to the complete archive.

WebDriver BiDi

Version 2.0.0 adds WebDriver BiDi without replacing WebDriver Classic. The HTTP session is created normally with the W3C `webSocketUrl = true` capability, then the client connects to the URL returned by the driver. Classic and BiDi commands therefore control **the same browser and the same session**.

Negotiation and direct client

```lua
local webdriver = require("webdriver")

local driver = assert(webdriver.firefox({
    headless = true,
    bidi = true,
}))

print(assert(driver:websocket_url()))
```

```lua
local bidi = assert(driver:bidi())
print(assert(bidi:status()).ready)
```

`bidi = true` requires Babet 2.22.0 or newer and `babet.websocket`. If the driver accepts session creation but does not return a usable `webSocketUrl`, babet-webdriver fails creation and cleans up the session, driver process and temporary profile.

Connection options accepted by `driver:bidi(options)`:

Option	Default	Description
`timeout`	5 s	WebSocket connection timeout
`command_timeout`	30 s	BiDi command budget
`close_timeout`	5 s	close-handshake budget
`event_queue_limit`	1024	maximum queued events
`verify`	Babet	TLS verification for `wss://`
`ca_cert`, `ca_path`, `hostname`, `min_version`	Babet	TLS parameters passed to the transport
`max_message_bytes`, `max_frame_bytes`	Babet	transport size limits

Typed commands

The 2.0.0 core surface includes:

```lua
local status = assert(bidi:status())
local tree = assert(bidi:get_tree({ max_depth = 0 }))
local context = tree.contexts[1].context

local navigation = assert(bidi:navigate(
    context,
    "https://example.com",
    { wait = "complete" }
))

local evaluation = assert(bidi:evaluate("document.title", context))
local realms = assert(bidi:get_realms({ context = context }))
```

`evaluate()` accepts either a browsing-context id or a `target` table. Supported options are `await_promise`, `result_ownership`, `serialization_options` and `user_activation`.

The BiDi standard evolves independently from babet-webdriver releases. For a command that does not yet have a convenience wrapper, `call()` provides the low-level escape hatch:

```lua
local result = assert(bidi:call("browser.getUserContexts", {}))
```

Subscriptions and events

```lua
local subscription = assert(bidi:subscribe({
    "log.entryAdded",
    "network.beforeRequestSent",
}, {
    contexts = { context },
}))

local event, err, code = bidi:next_event(5)
if event then
    print(event.method)
elseif code ~= "timeout" then
    error(err)
end

assert(bidi:unsubscribe(subscription))
```

Events that arrive while a command response is pending are retained and never desynchronize command ids. The queue is bounded by `event_queue_limit`. When it overflows, excess events are counted and the synthetic `babetWebDriver.eventOverflow` event reports the loss.

An explicit callback interface is also available:

```
bidi:on("log.entryAdded", function(event)
    print(event.params.text)
end)

assert(bidi:dispatch(5, 10))
```

`dispatch()` runs callbacks in the calling Lua thread. No implicit concurrent callback can therefore interrupt an arbitrary user command.

Dedicated BiDi worker

`driver:bidi_worker()` moves the WebSocket connection into a separate worker:

```
local driver = assert(webdriver.chromium({ headless = true, bidi = true }))
local bidi = assert(driver:bidi_worker({ event_queue_limit = 1024 }))
local subscription = assert(bidi:subscribe("log.entryAdded"))

local context = assert(bidi:get_tree({ max_depth = 0 })).contexts[1].context
assert(bidi:evaluate("console.log('hello bidi')", context))
local event = assert(bidi:next_event(5))
print(event.method)

assert(bidi:unsubscribe(subscription))
assert(driver:quit())
```

For a Classic session already hosted by `webdriver_worker.lua`, the parent only needs to call `session:bidi()`. The BiDi worker is separate from the Classic worker and owns its own WebSocket.

Because `babet.websocket` is synchronous, this worker is command-driven: it waits on its parent channel and reads the WebSocket only while processing a BiDi command or `next_event()`. A long `next_event(timeout)` is read in bounded slices so worker cancellation remains responsive. It never polls the network while idle, preventing a WebSocket `recv()` from starving the next command. Events interleaved with a command are retained in the bounded BiDi client queue (`event_queue_limit`).

Worker-specific options are `channel_capacity`, `command_timeout`, `worker_start_timeout` and `stop_timeout`. BiDi client options, including `event_queue_limit`, are also accepted and forwarded to the transport.

`driver:quit()` and `session:stop()` automatically close an attached BiDi transport before ending the Classic session. BiDi objects also support Lua 5.4/5.5 to-be-closed variables through `__close`.

For `webdriver_worker`, `stop_timeout` (or the explicit `timeout` passed to `session:stop(timeout)`) is a global budget: it covers closing any attached BiDi worker first, then stopping and joining the Classic worker.

Workers and channels

`webdriver_worker.lua` keeps the session and driver process inside a persistent worker.

```
local worker_driver = require("webdriver_worker")
local session = assert(worker_driver.start({
    browser = "firefox",
    headless = true,
    channel_capacity = 64,
    command_timeout = 120,
```

```
}))

assert(session:open("https://example.com"))
local heading = assert(session:css("h1"))
print(assert(heading:text()))
assert(session:stop())
```

The parent response budget covers at least `request_timeout` plus a small transport margin. For `wait()`, it also covers the requested logical timeout and one final HTTP request, so the proxy cannot expire before its worker.

Before creating the worker, the parent prepares the driver when necessary. Concurrent installations remain safe: every extraction uses its own temporary file and the final executable is published atomically.

Element and Shadow DOM proxies

A WebDriver element is not sent as userdata. The worker sends its W3C id and the parent creates a proxy:

```
local element = assert(session:css("h1"))
assert(worker_driver.is_element(element))
print(element:element_id())
```

A `ShadowRoot` follows the same model:

```
local shadow = assert(element:shadow_root())
assert(worker_driver.is_shadow_root(shadow))
local button = assert(shadow:find("button"))
```

Proxies can be reused as arguments to `js()` or `frame()`. The internal transport also preserves `nil` arguments, including in the middle of an argument list, multiple return values, and the W3C error code.

Lifecycle

```
print(session:status())
assert(session:stop(10))
```

`stop()` first asks the worker to close the session cleanly. On timeout, Babet cancellation is requested. Cancellation remains cooperative, so every HTTP operation in the module is bounded.

A proxy session is synchronous and must have only one command in flight. Create multiple sessions for parallel execution.

The chainable `actions()` builder is not transported through the worker proxy. Chainable actions remain available on a direct session.

Error contract

An operational error returns:

```
nil, "webdriver: ..."
```

A WebDriver response preserves its W3C code in the message and, for methods that expose it such as `find()`, as a third return value:

```
local element, err, code = driver:find("#missing")
assert(element == nil and code == "no such element")
```

Example messages:

```
webdriver: no such element: element missing
webdriver: stale element reference: ...
webdriver: invalid session id: ...
```

Programming errors that raise a Lua exception include:

- unknown option;
- invalid type;
- sparse argument array;
- unknown locator strategy;
- invalid frame type;
- cyclic table passed to JavaScript.

Limitations

- the direct BiDi client is synchronous; use `bidi_worker()` to isolate the WebSocket loop;
- the typed 2.0.0 BiDi surface covers the session/browsingContext/script core; `call()` exposes the rest of the protocol;
- automatic driver management currently supports Linux only;
- Chrome for Testing provides a Linux x86-64 binary only;
- no authenticated or TLS remote proxy for the WebDriver server;
- no download resume;
- no progress callback;
- worker proxy limited to serializable methods and without the Actions builder;
- Babet channels carry JSON-like data, not binary strings containing NUL bytes.

WebDriver session in a worker

The complete integration path can be verified with:

```
HEADLESS=1 ./run_worker_smoke.sh firefox
HEADLESS=1 ./run_worker_smoke.sh chromium
```

This test covers the parent process, channels, worker, driver process and real browser.